

Abstraction in Java

Abstraction is one of the four fundamental principles of Object-Oriented Programming (OOP), alongside Encapsulation, Inheritance, and Polymorphism. It focuses on hiding the implementation details of a system and showing only the essential features or functionality to the user.

In Java, abstraction allows developers to define a class or interface that provides a blueprint for specific behavior without detailing the implementation.

Definition

Abstraction is a process of hiding the internal details and showing only essential information to the user. For example, when you drive a car, you interact with the steering wheel, accelerator, and brake, but you don't need to know how the engine works internally.

Why Use Abstraction?

1. **Improved Code Maintainability:** By separating the implementation from the interface, abstraction ensures that changes to the implementation do not affect code that uses the interface.
 2. **Enhanced Security:** Users can't access internal implementation details, reducing the chances of unintended manipulation.
 3. **Modularity:** Abstraction promotes modular development by breaking down the program into smaller, more manageable pieces.
 4. **Focus on Design:** Developers can focus on what the system does rather than how it does it.
-

Levels of Abstraction

1. **Abstract Classes:**
 - Can have both abstract methods (methods without a body) and concrete methods (methods with a body).
 - Used when some methods have common implementation across subclasses, while others are defined differently in each subclass.
2. **Interfaces:**

- Provide 100% abstraction in Java (prior to Java 8).
 - Defines a contract that the implementing classes must follow.
-

1. Abstract Classes

An **abstract class** in Java:

- Is declared using the `abstract` keyword.
- Cannot be instantiated directly.
- May contain abstract methods (methods without implementation) and non-abstract methods (methods with implementation).

Syntax:

```
abstract class ClassName {  
    // Abstract method  
    abstract void methodName();  
  
    // Concrete method  
    void anotherMethod() {  
        System.out.println("This is a concrete method.");  
    }  
}
```

Example:

```
abstract class Shape {  
    // Abstract method  
    abstract void draw();  
  
    // Concrete method  
    void display() {  
        System.out.println("This is a shape.");  
    }  
}  
  
class Circle extends Shape {  
    @Override  
    void draw() {  
        System.out.println("Drawing a Circle");  
    }  
}  
  
class Rectangle extends Shape {  
    @Override
```

```

    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

public class AbstractClassExample {
    public static void main(String[] args) {
        Shape shape;

        shape = new Circle();
        shape.draw(); // Calls Circle's implementation
        shape.display();

        shape = new Rectangle();
        shape.draw(); // Calls Rectangle's implementation
        shape.display();
    }
}

```

Output:

```

Drawing a Circle
This is a shape.
Drawing a Rectangle
This is a shape.

```

2. Interfaces

An **interface** in Java:

- Is a collection of abstract methods.
- Can also include static methods, default methods, and constants (since Java 8).
- Is implemented by a class using the `implements` keyword.

Syntax:

```

interface InterfaceName {
    // Abstract method
    void methodName();
}

```

Example:

```

interface Animal {
    void sound(); // Abstract method
}

class Dog implements Animal {
    @Override
    public void sound() {
        System.out.println("Dog barks");
    }
}

class Cat implements Animal {
    @Override
    public void sound() {
        System.out.println("Cat meows");
    }
}

public class InterfaceExample {
    public static void main(String[] args) {
        Animal animal;

        animal = new Dog();
        animal.sound(); // Calls Dog's implementation

        animal = new Cat();
        animal.sound(); // Calls Cat's implementation
    }
}

```

Output:

```

Dog barks
Cat meows

```

Difference Between Abstract Class and Interface

Feature	Abstract Class	Interface
Methods	Can have both abstract and concrete methods.	Before Java 8: Only abstract methods. After Java 8: Can have abstract, static, and default methods.
Fields	Can have instance variables and static fields.	Can have only static and final fields (constants).

Feature	Abstract Class	Interface
Multiple Inheritance	A class can extend only one abstract class.	A class can implement multiple interfaces.
Constructors	Can have constructors.	Cannot have constructors.
Access Modifiers	Can have any access modifier (public, protected, private).	All methods are implicitly <code>public</code> .

Advantages of Abstraction

1. **Hides Complexity:** Users only see the relevant details, not the implementation.
 2. **Code Reusability:** Common functionality can be abstracted into a superclass or interface.
 3. **Improves Modularity:** Each subclass or implementing class focuses on its specific behavior.
 4. **Better Security:** Implementation details remain hidden from unauthorized access.
-